

MQTT – Einführung

Vom Sensor zum Subscriber · Konzepte, Protokoll, Praxis

Dieses Dokument ist:

- Skript für einen 15-minütigen Einstiegsvortrag
- Selbststudium-Text wenn ihr das alleine durcharbeitet
- Referenz für das MQTT-Labor

Was ihr danach versteht:

- Was MQTT ist und warum es existiert
- Was Publish/Subscribe bedeutet und wie es sich von anderen Ansätzen unterscheidet
- Was QoS ist und welches Level wir im Labor warum verwenden
- Was persistent sessions, retained messages und seq-Nummern bedeuten – und wer für was zuständig ist
- Was passiert wenn etwas ausfällt



Zum Selbststudium

Falls ihr dieses Dokument ohne Vortrag lest: Alle Konzepte sind hier vollständig erklärt. Die **farbigen Boxen** heben das Wichtigste hervor. Plant etwa 30–35 Minuten Lesezeit ein.

1

Das Problem – warum nicht einfach senden?

Stellt euch vor: Auf dem Campus gibt es drei Wetterstationen – S1, S2, S3. Jede misst alle paar Sekunden Temperatur und Luftfeuchtigkeit. Diese Daten sollen:

- **live angezeigt** werden – auf einem Dashboard im Eingangsbereich
- **ausgewertet** werden – von einer Analyse-App die Ausreißer erkennt
- **gespeichert** werden – in einer Datenbank für historische Vergleiche
- **alarmiert** werden – per E-Mail wenn ein Wert kritisch wird

Die naheliegende Lösung: Jede Station sendet ihre Daten direkt an jeden Empfänger.

Direkte Verbindungen – das naive Modell:

S1 → Dashboard, S1 → Analyse-App, S1 → Datenbank, S1 → Alarm-Service

S2 → Dashboard, S2 → Analyse-App, S2 → Datenbank, S2 → Alarm-Service

S3 → Dashboard, S3 → Analyse-App, S3 → Datenbank, S3 → Alarm-Service

Ergebnis: $3 \times 4 = 12$ Verbindungen. Und das bei nur 3 Stationen und 4 Empfängern. Kommt eine fünfte App dazu, muss jede der 3 Stationen ihre Verbindungslogik anpassen.

**Das eigentliche Problem**

Jede Station **muss wissen, wer ihre Daten haben will**. Und jeder Empfänger muss wissen, von welchen Stationen er Daten kriegt. Beide Seiten sind aneinander gekoppelt – ändert sich eine, muss die andere reagieren.

**Analogie aus dem Alltag**

Stellt euch vor, ihr schreibt einen Artikel für die Campuszeitung. Direkte Verbindungen würden bedeuten: ihr müsstet jeden Studierenden persönlich anrufen der euren Artikel lesen will. Und ihr müsstet jedesmal alle anrufen wenn ihr einen neuen Artikel schreibt. Stattdessen gibt es die **Zeitung** – ihr gebt euren Artikel ab, und wer ihn lesen will, holt sich die Zeitung. Ihr kennt eure Leser nicht, und eure Leser kennen euch nicht direkt.

2

MQTT und das Publish/Subscribe-Prinzip

MQTT löst genau dieses Problem mit einem Vermittler: dem Broker.

**Merksatz**

Publisher und Subscriber kennen sich nicht. Der Publisher weiß nicht, wer seine Nachrichten liest. Der Subscriber weiß nicht, wer sie schreibt. Beide kennen nur den Broker und das Topic. Das ist **Entkopplung** – und das ist der ganze Witz.

Was ist ein Topic?

Ein Topic ist eine Adresse – eine hierarchische Zeichenkette die beschreibt, was eine Nachricht ist. Wie ein Ordnerpfad, nur für Nachrichten:

Topic-Beispiele aus unserem Labor

```

campus/station-01/environment
campus/station-02/environment
campus/station-03/environment
  
```

```

campus/+/environment
campus/#
  
```

← Wildcard + : genau eine Ebene
 ← Wildcard # : alles darunter

Die Hierarchie ist frei erfindbar – MQTT schreibt kein Format vor. Aber eine gute Hierarchie folgt dem Muster: Ort → Gerät → Messgröße. So können Subscriber präzise filtern:

- Das Dashboard will alle Stationen: abonniert `campus/+/environment`
- Der Alarm-Service will nur S1: abonniert `campus/station-01/environment`
- Die Datenbank will alles: abonniert `campus/#`

MQTT vs. andere Ansätze – ein Vergleich

Ansatz	Funktionsweise	Stärken	Schwächen / wann ungeeignet
Direkte TCP-Verbindung	A sendet direkt an B's IP:Port	Einfach, keine Infrastruktur	Sender muss Empfänger kennen. Skaliert nicht.

			Fällt B aus, verliert A Daten.
HTTP REST (Request/Response)	Client fragt, Server antwortet	Weit verbreitet, Firewalls freundlich	Pull-Modell: Client muss aktiv anfragen. Für kontinuierliche Datenströme ineffizient.
WebSocket	Bidirektionaler Kanal, Browser-freundlich	Gut für Echtzeit im Browser	Kein eingebautes Topic-System. Kein QoS. Kein Pub/Sub von Haus aus.
MQTT ✓	Publisher → Broker → Subscriber, Topic-basiert	Entkopplung. QoS-Garantien. Minimal-Overhead (2 Byte Header). Läuft auf Mikrocontrollern.	Broker ist Single Point of Failure. Kein vollständiges Request/Response.



In der Praxis

Wo MQTT in der Praxis überall steckt:

- IoT-Geräte: Smarthome-Systeme (Philips Hue, Home Assistant), Industriesensorik
- Fahrzeugtechnik: Telemetriedaten in Echtzeit von Fahrzeugen an Backend
- Medizintechnik: Patientenmonitoring – Sensor sendet, mehrere Systeme empfangen
- Logistik: Pakettracking – Tausende Scanner publizieren, zentrale Systeme subscriben
- **Nicht sinnvoll:** klassische Web-APIs, Datenbankabfragen, Dateiübertragung – dafür gibt es bessere Werkzeuge.

3 Was steckt in einer Nachricht?

Eine MQTT-Nachricht besteht aus exakt zwei Teilen: dem Topic und dem Payload. Mehr nicht.

Teil	Bedeutung
Topic	Die Adresse der Nachricht – wohin sie gehört. Reiner Text, z.B. <code>campus/station-01/environment</code> . Der Broker nutzt das Topic zum Weiterleiten.
Payload	Der eigentliche Inhalt – beliebige Bytes. MQTT interessiert sich nicht dafür was drin steht. Wir verwenden JSON, weil es lesbar und sprachunabhängig ist.

In unserem Labor sieht eine vollständige Nachricht so aus:

Beispiel-Nachricht aus unserem Labor

Topic: `campus/station-01/environment`

```
Payload: {  
  "seq": 42,  
  "station_id": "S1",  
  "timestamp": "2024-03-01T08:15:32Z",  
  "temperature_c": 19.3,  
  "humidity_pct": 64.2  
}
```



Warum JSON?

MQTT selbst ist formatagnostisch – der Payload könnte auch ein einzelnes Byte sein, CSV-Text, oder Binärdaten. Wir verwenden JSON weil:

- es für Menschen lesbar ist – ihr könnt im Terminal sehen was gesendet wird
- jede Sprache es parsen kann – wenn ihr später Python oder Java schreibt, ist der gleiche Parser nutzbar
- es selbstdokumentierend ist – `"temperature_c"` erklärt sich selbst, `"19.3"` allein nicht

4

Quality of Service – was passiert wenn etwas ausfällt?

Netzwerke sind unzuverlässig. Pakete gehen verloren, Verbindungen brechen ab, Prozesse stürzen ab. MQTT adressiert das mit dem QoS-Level – einer Vereinbarung zwischen Publisher, Broker und Subscriber darüber, wie zuverlässig eine Nachricht zugestellt werden soll.

Es gibt drei Stufen:

Level	Name	Garantie	Overhead	Einsatz im Labor
QoS 0	At most once	Keine – fire and forget	Minimal	Nie: zu riskant für Sensordaten
QoS 1 ✓	At least once	Mindestens 1× zugestellt (Duplikat möglich)	1 ACK	Unser Lab: Publisher wartet auf PUBACK
QoS 2	Exactly once	Genau 1× – kein Duplikat	4-Wege-Handshake	Selten: zu teuer für schnelle Sensordaten



Analogie aus dem Alltag

Die Post-Analogie:

- **QoS 0:** Postwurf-Sendung. Wird eingeworfen und vergessen. Ankommt was ankommt.
- **QoS 1:** Einschreiben. Empfänger muss Eingang bestätigen (PUBACK). Kommt die Bestätigung nicht, wird nochmal gesendet – daher können Duplikate entstehen.
- **QoS 2:** Einschreiben mit Rückschein und Protokoll. Genau eine Zustellung garantiert, aber viermal so aufwändig.

Was genau passiert bei QoS 1 in unserem Labor?

1 PUBLISH	Publisher sendet Nachricht an Broker: PUBLISH (QoS 1, Message-ID: 42)
2 PUBACK	Broker empfängt und speichert die Nachricht, sendet PUBACK (Message-ID: 42) zurück an Publisher
3 WEITERLEITEN	Broker leitet Nachricht an alle Subscriber weiter die das Topic abonniert haben
4 PUBACK	Jeder Subscriber sendet PUBACK an Broker. Erst dann gilt die Nachricht als vollständig zugestellt

**Was passiert wenn Schritt 2 ausbleibt?**

Kommt das PUBACK nicht zurück (Netzwerkproblem, Broker kurz überlastet), sendet der Publisher die Nachricht nochmal – mit dem DUP-Flag gesetzt. Deshalb kann eine Nachricht bei QoS 1 doppelt ankommen. Euer Validator sollte die **seq-Nummer** nutzen um Duplikate zu erkennen. Das ist bewusst Teil eurer Spec-Aufgabe.

Warum QoS 1 im Labor – und nicht 2?

QoS 2 würde Duplikate vollständig verhindern – aber es erfordert vier Nachrichten pro Übertragung statt zwei. Bei Sensordaten die alle zwei Sekunden ankommen und wo ein gelegentliches Duplikat problemlos filterbar ist, ist das Overhead ohne echten Nutzen. In der Industrie ist QoS 1 + applikationsseitige Duplikatserkennung (via seq-Nummer) die übliche Lösung.

5 Zustand und Gedächtnis – wer erinnert sich an was?

QoS regelt ob eine einzelne Nachricht ankommt. Aber was ist mit dem größeren Bild – was passiert mit dem Zustand des Systems wenn Komponenten aus- und wieder einsteigen? Dafür gibt es drei Konzepte die zusammenspielen aber voneinander unabhängig sind.

5.1 – Die seq-Nummer: euer eigenes Gedächtnis

Die seq-Nummer im JSON-Schema ist kein MQTT-Konzept. MQTT weiß nicht dass sie existiert. Sie ist etwas das ihr selbst eingebaut habt – und das ist genau der Punkt.

Was der Broker weiß:

- Nachrichten kamen an und wurden zugestellt
- Wie viele Subscriber es gibt
- Welche Topics aktiv sind
- **Nicht:** ob Nachrichten logisch zusammenhängen
- **Nicht:** ob eine Sequenz lückenlos ist
- **Nicht:** ob eine Nachricht ein Duplikat ist

Was die seq-Nummer euch gibt:

- Lückenerkennung: seq springt von 15 auf 23 → 7 Nachrichten fehlen
- Duplikaterkennung: seq=42 kommt zweimal → QoS-1-Duplikat ignorieren
- Neustart-Erkennung: seq fängt bei 1 an → Publisher wurde neu gestartet
- Reihenfolge: seq ist aufsteigend → Nachrichten sind in der richtigen Reihenfolge



Merksatz

Die seq-Nummer ist **Anwendungslogik, nicht Protokolllogik**. Der Broker liefert Nachrichten – was sie bedeuten, ob sie vollständig sind, ob eine fehlt: das wisst nur ihr. Deshalb steht seq im Schema, nicht im MQTT-Header.

5.2 – Persistent Sessions: das Gedächtnis des Brokers

Wenn ein Subscriber die Verbindung verliert – absichtlich oder durch Fehler – stellt sich die Frage: was passiert mit Nachrichten die während seiner Abwesenheit ankommen? Die Antwort hängt davon ab, ob eine persistent session verwendet wird.



Analogie aus dem Alltag

Stellt euch vor, ihr abonniert eine Tageszeitung. **Ohne persistent session:** seid ihr eine Woche im Urlaub, sind die Ausgaben weg – niemand hat sie für euch aufgehoben. **Mit persistent session:** der Zusteller legt alle Ausgaben in euer Fach und ihr bekommt sie nach der Rückkehr alle auf einmal.

Einstellung	clean_session = true (unser Labor)	clean_session = false (Produktion)
Subscriber trennt sich	Session wird sofort gelöscht	Session bleibt beim Broker erhalten

Nachrichten während Ausfall	Werden verworfen – niemand da zum Empfangen	Broker puffert alle QoS 1+ Nachrichten
Subscriber reconnected	Startet komplett neu – keine verpassten Nachrichten	Broker liefert alle gepufferten Nachrichten nach
Konfiguration in C	<code>mosquitto_new("id", true, NULL)</code>	<code>mosquitto_new("id", false, NULL)</code>
Wann sinnvoll	Labs, Tests, einfache Setups	Wenn Datenverlust inakzeptabel ist

**Achtung**

Persistent sessions funktionieren nur mit QoS 1 oder 2. QoS 0 Nachrichten werden nie gepuffert – egal was `clean_session` sagt. Außerdem: der Broker puffert nur für Clients mit einer bekannten Client-ID. Wechselt die Client-ID bei jedem Connect, gibt es keine Session zu erinnern.

5.3 – Retained Messages: das Gedächtnis für neue Subscriber

Persistent sessions lösen das Problem für bekannte Subscriber die kurz weg waren. Aber was ist mit einem brandneuen Subscriber der sich zum ersten Mal verbindet? Er hat keine Session, also gibt es nichts zu puffern. Trotzdem möchte er sofort den aktuellen Zustand kennen – und nicht warten bis die nächste Nachricht zufällig eintrifft.

**Analogie aus dem Alltag**

Ein schwarzes Brett in der Universität. Wer es das erste Mal sieht, liest den aktuell angehefteten Zettel – unabhängig davon, wann er angeheftet wurde. Ein neuer Subscriber bekommt den letzten retained Wert sofort, ohne warten zu müssen.

Wie Retained Messages funktionieren:

- Publisher sendet eine Nachricht mit `retain = true` (letzter Parameter in `mosquitto_publish`)
- Broker speichert diese Nachricht pro Topic – immer nur die letzte
- Jeder neue Subscriber bekommt die retained Message sofort nach dem Subscribe
- Eine neue retained Message überschreibt die alte – pro Topic gibt es genau eine
- Retained Message löschen: leere Nachricht mit `retain = true` senden

In unserem Labor:

- Unser Publisher sendet mit `retain = false` – bewusste Entscheidung für den Einstieg
- Konsequenz: startet ihr den Subscriber nach dem Publisher, wartet er auf die nächste Nachricht
- Mit `retain = true`: der neue Subscriber würde sofort den letzten Wert jeder Station sehen

Problem	Lösung	Wer konfiguriert es?
Subscriber verpasst Nachrichten weil er zu spät startet	Persistent session (<code>clean_session = false</code>)	Client – bei <code>mosquitto_new()</code>
Neuer Subscriber kennt aktuellen Zustand nicht	Retained Messages (<code>retain = true</code>)	Publisher – bei <code>mosquitto_publish()</code>

Duplikate durch QoS-1-Wiederholung	seq-Nummer im Schema, Applikationslogik	Eure Anwendung – kein Broker-Feature
Lücken in der Sequenz erkennen	seq-Nummer im Schema, Validator	Eure Anwendung – kein Broker-Feature

6

Was passiert wenn etwas ausfällt?

Das ist die Frage die Phase 1 eures Labors beantwortet – aber hier ist die Theorie dahinter, damit ihr die richtigen Antworten geben könnt.

Szenario 1: Subscriber fällt aus

Ohne persistent session (unser Labor)

- Subscriber trennt sich
- Publisher sendet weiter an Broker
- Broker hat niemanden zum Weiterleiten
- Nachrichten werden verworfen
- **Subscriber reconnected:** bekommt nur Nachrichten ab jetzt

Mit persistent session (Produktion)

- Subscriber trennt sich – Session bleibt beim Broker
- Broker puffert alle QoS 1+ Nachrichten
- **Subscriber reconnected:** Broker liefert alle verpassten Nachrichten nach
- Funktioniert auch nach Stunden oder Tagen

**Achtung**

Im Labor nutzen wir **keine persistent session**. Das bedeutet: wenn ihr den Subscriber startet, **nachdem** der Publisher schon Nachrichten gesendet hat, sind diese verloren. Startet immer zuerst den Subscriber, dann den Publisher.

Szenario 2: Publisher fällt aus

Das ist das einfachere Szenario. Fällt der Publisher aus:

- Der Broker bemerkt das nach Ablauf des Keepalive-Timeouts (in unserem Lab: 60 Sekunden)
- Der Broker trennt die Verbindung und gibt Ressourcen frei
- Subscriber erhalten keine neuen Nachrichten – aber sie stürzen nicht ab
- Reconnectet der Publisher, **fängt er bei der nächsten seq-Nummer an** – der Subscriber sieht die Lücke in den seq-Nummern

**Die seq-Nummer als Diagnosewerkzeug**

Genau deshalb hat euer Schema eine **seq-Nummer**: Sie ist nicht nur zur Duplikaterkennung, sondern auch um Lücken zu sehen. Seq springt von 15 auf 23? Publisher war 8 Nachrichten lang offline. Das ist Information – nicht nur ein Fehler.

Szenario 3: Broker fällt aus

Das ist das kritischste Szenario – der Broker ist das Herzstück des Systems:

- Publisher kann sich nicht verbinden – `mosquitto_connect()` gibt Fehler zurück

- Subscriber verliert die Verbindung – on_disconnect wird aufgerufen
- Solange der Broker unten ist, kommen keine Nachrichten durch
- Nach Broker-Neustart: Publisher und Subscriber müssen sich neu verbinden (reconnect)



In der Praxis

Wie die Industrie das löst:

- Broker-Clustering: mehrere Broker-Instanzen, Failover automatisch
- Retained Messages: letzte Nachricht bleibt beim Broker gespeichert – neuer Subscriber bekommt sofort den letzten Wert
- Last Will and Testament: Publisher hinterlegt beim Connect eine Nachricht die der Broker automatisch sendet wenn er ausfällt
- **In unserem Labor:** Broker läuft lokal, kein Failover – aber ihr seht den Fehlerfall direkt wenn ihr mosquitto beendet während Publisher läuft.

Szenario 4: Netzwerkausfall (kurz)

Ein kurzer Ausfall (unter dem Keepalive-Timeout von 60 Sekunden) bleibt für libmosquitto oft transparent, wenn mosquitto_loop_forever() läuft:

- mosquitto_loop_forever() versucht automatisch zu reconnecten
- Nachrichten die während des Ausfalls gesendet wurden gehen je nach QoS verloren oder werden bei Reconnect nochmal gesendet
- on_disconnect und on_connect feuern – euer Code kann darauf reagieren

7

Das große Bild – wo wir im Labor stehen

Hier ist, wie das was ihr im Labor baut, in das Gesamtsystem passt – und was in späteren Veranstaltungen dazu kommt:

Jetzt: C-Labor	Bald: Arduino-Labor	Später: Python/Java	Produktion
Datengenerator simuliert Sensor	Echter Sensor liefert Daten	Echter Subscriber verarbeitet Daten	Produktiver Stack mit Failover
JSON-Serializer serialize_to_json()	gleicher Serializer auf Arduino	JSON-Parser in Python/Java	Schema-Versionierung Rückwärtskompatibilität
Validator validate_message()	Validator läuft auf Subscriber-Seite	Validator in Python/Java	Monitoring Alerting

**Merksatz**

Was ihr jetzt in C spezifiziert – das Schema, die Felder, die Wertebereiche – wird in jeder späteren Schicht gelten. **Das Schema ist der Vertrag des Systems.** Nicht der Code.

Was ihr jetzt mitgenommen haben solltet

✓	MQTT ist ein Nachrichtenprotokoll mit einem Vermittler (Broker). Publisher und Subscriber kennen sich nicht – nur das Topic.
✓	Ein Topic ist eine hierarchische Adresse. Wildcards (+ und #) erlauben flexibles Abonnieren.
✓	QoS bestimmt die Zustellgarantie. QoS 1 ist der sinnvolle Kompromiss für Sensordaten: Zustellung garantiert, Duplikate möglich.
✓	Die seq-Nummer ist Anwendungslogik – der Broker kennt sie nicht. Sie erkennt Lücken, Duplikate und Neustarts.
✓	Persistent session (clean_session = false): der Broker puffert verpasste Nachrichten für einen bekannten Subscriber. Konfiguriert vom Client.
✓	Retained message (retain = true): der Broker merkt sich die letzte Nachricht pro Topic. Neue Subscriber bekommen sie sofort. Konfiguriert vom Publisher.
✓	Fällt der Subscriber aus, verliert er ohne persistent session alle Nachrichten bis zum Reconnect.
✓	Das Schema (Feldnamen, Typen, Wertebereiche) ist der einzige Vertrag zwischen Publisher und Subscriber.

**Zum Selbststudium**

Wenn ihr das alleine durchgearbeitet habt:

- Könnt ihr erklären, warum MQTT und nicht direkte TCP-Verbindungen?
- Könnt ihr beschreiben was passiert, wenn der Subscriber 10 Sekunden nach dem Publisher startet?
- Wisst ihr, warum die seq-Nummer im JSON nicht optional ist?
- Könnt ihr den Unterschied zwischen persistent session und retained message in einem Satz erklären?
- Wisst ihr, wer jeweils clean_session und retain konfiguriert – Client oder Publisher?

Falls nein: Lest den entsprechenden Abschnitt nochmal und stellt die Frage zu Beginn des Labors.

Ende der Einführung · Weiter geht es mit dem Arbeitsblatt